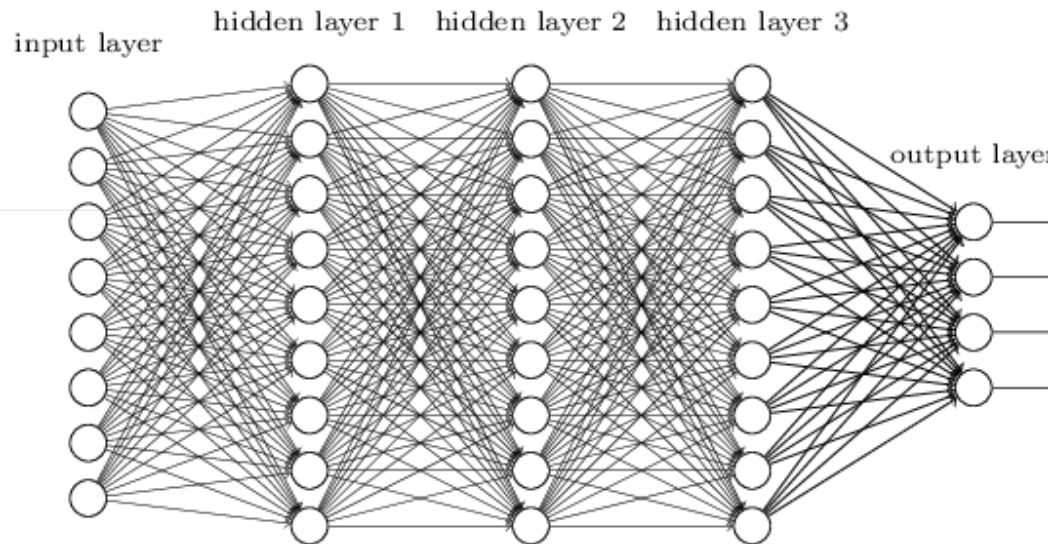
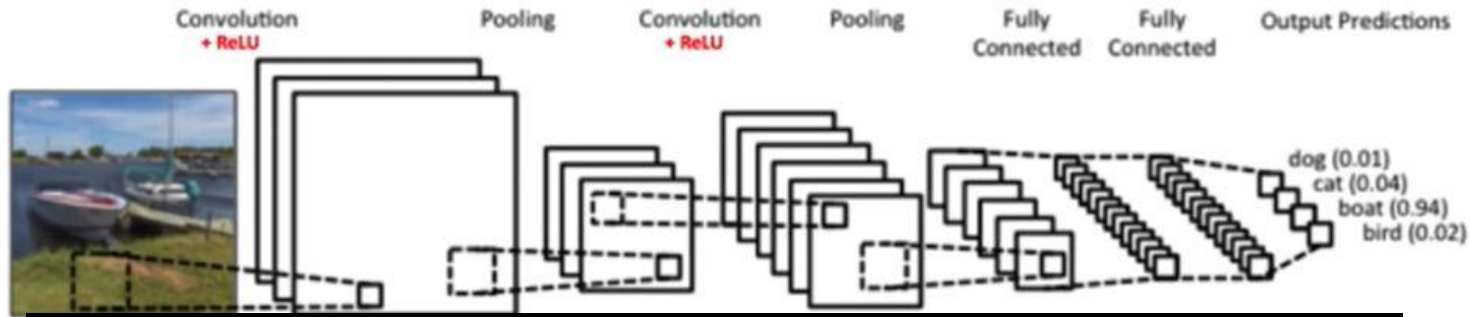


Smaller Network: CNN

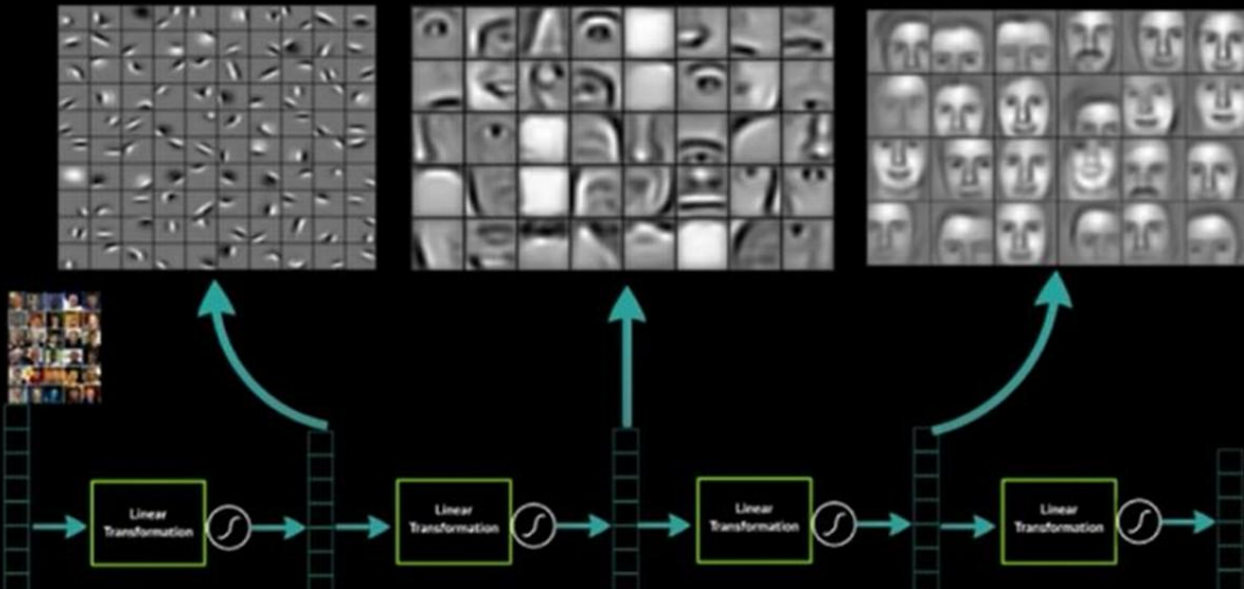
```
model=Sequential()  
model.add(Dense(input_dim=28*28, units=500,activation="sigmoid"))  
model.add(Dense(units=500, activation="sigmoid"))  
model.add(Dense(units=500, activation="sigmoid"))  
  
model.add(Dense(units=10, activation="softmax"))  
  
model.compile(loss=keras.losses.categorical_crossentropy,optimizer="adam", metrics=['accuracy'])  
  
model.fit(x_train, y_train, batch_size=100, epochs=2, verbose=1)  
  
score=model.evaluate(x_test,y_test, verbose=0)  
  
print("Test loss", score[0])  
print("Test accuracy", score[1])
```

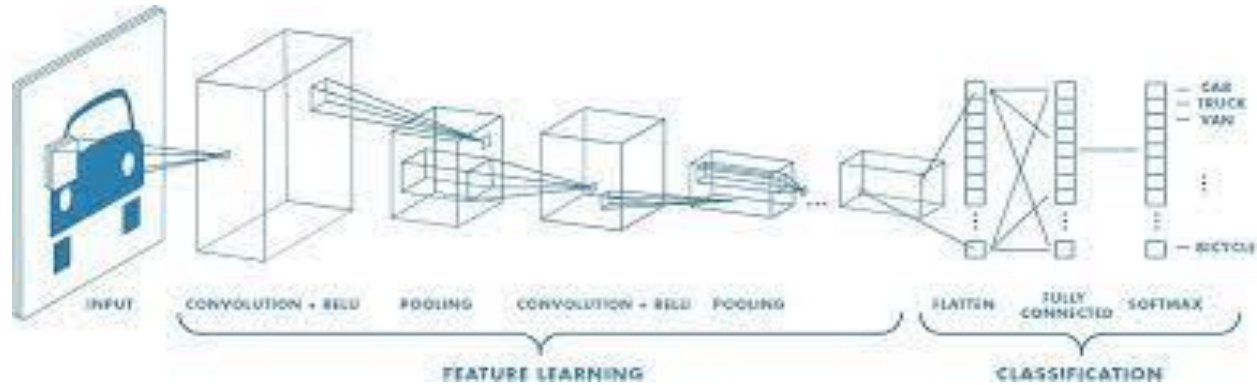
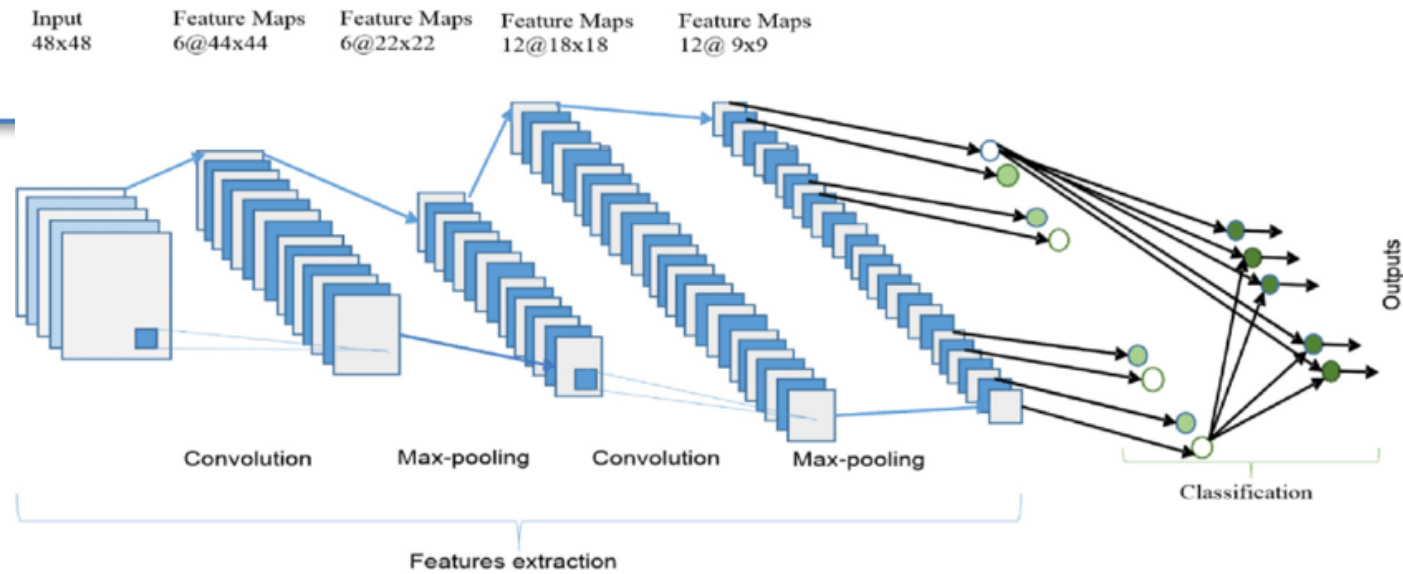


CNN Network



Deep Learning learns layers of features





Deep Learning



Residual Network and other CNN variants

ImageNet (2010)

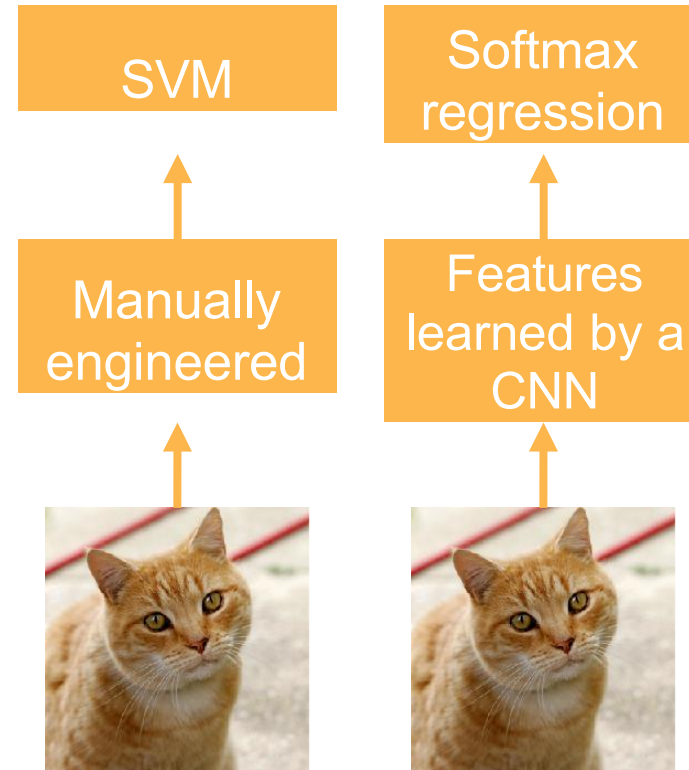


Images	Color images with nature objects	Gray image for hand-written digits
Size	469 x 387	28 x 28
#examples	1.2 M	60 K
# classes	1,000	10

AlexNet



- AlexNet won ImageNet competition in 2012
- Deeper and bigger LeNet
- Key modifications
 - Dropout (regularization)
 - ReLu (training)
 - MaxPooling
- Paradigm shift for computer vision

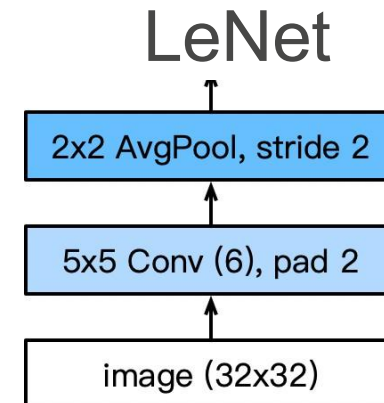
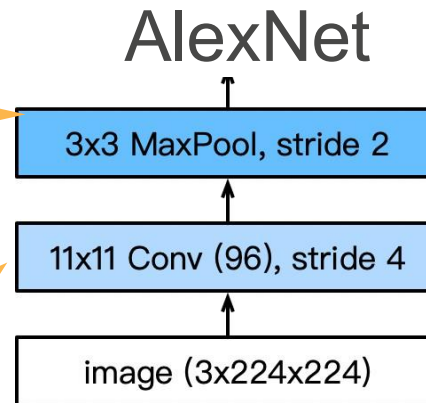


AlexNet Architecture

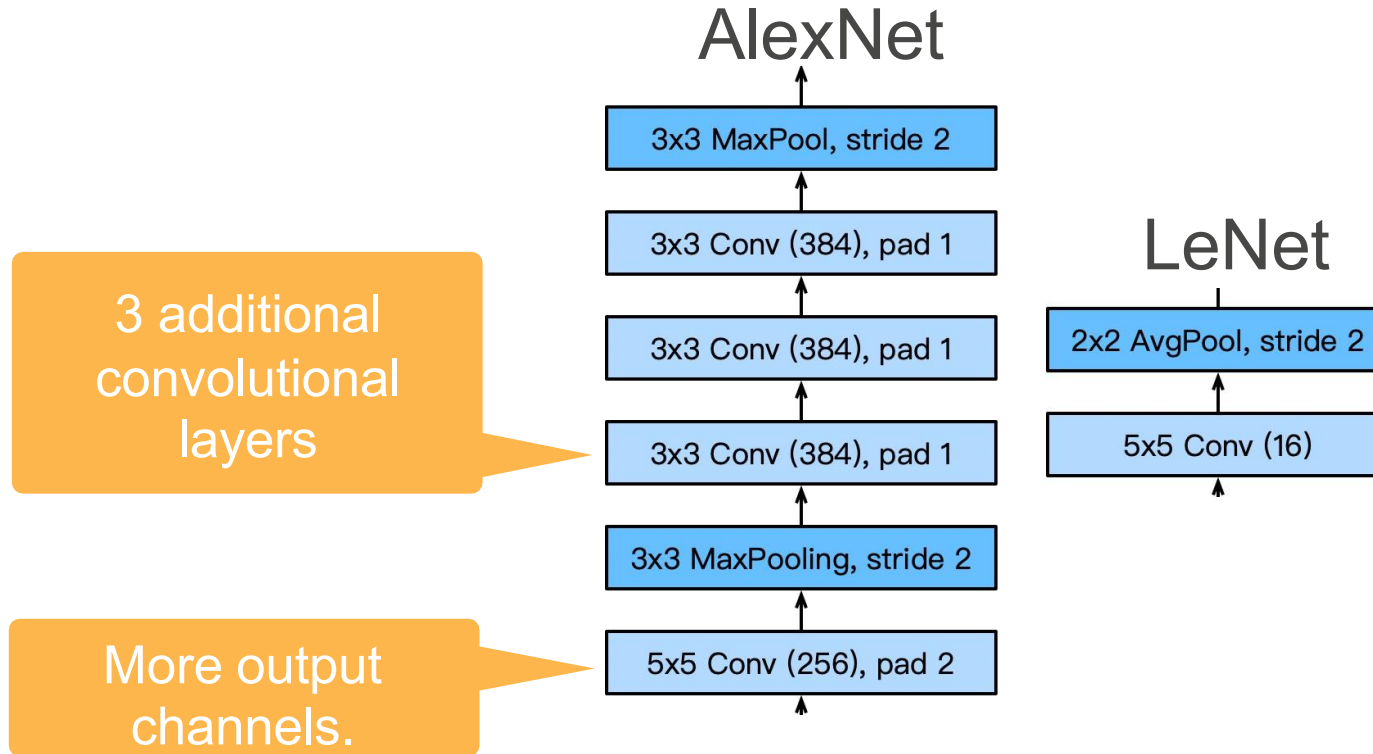


Larger pool size,
change to max pooling

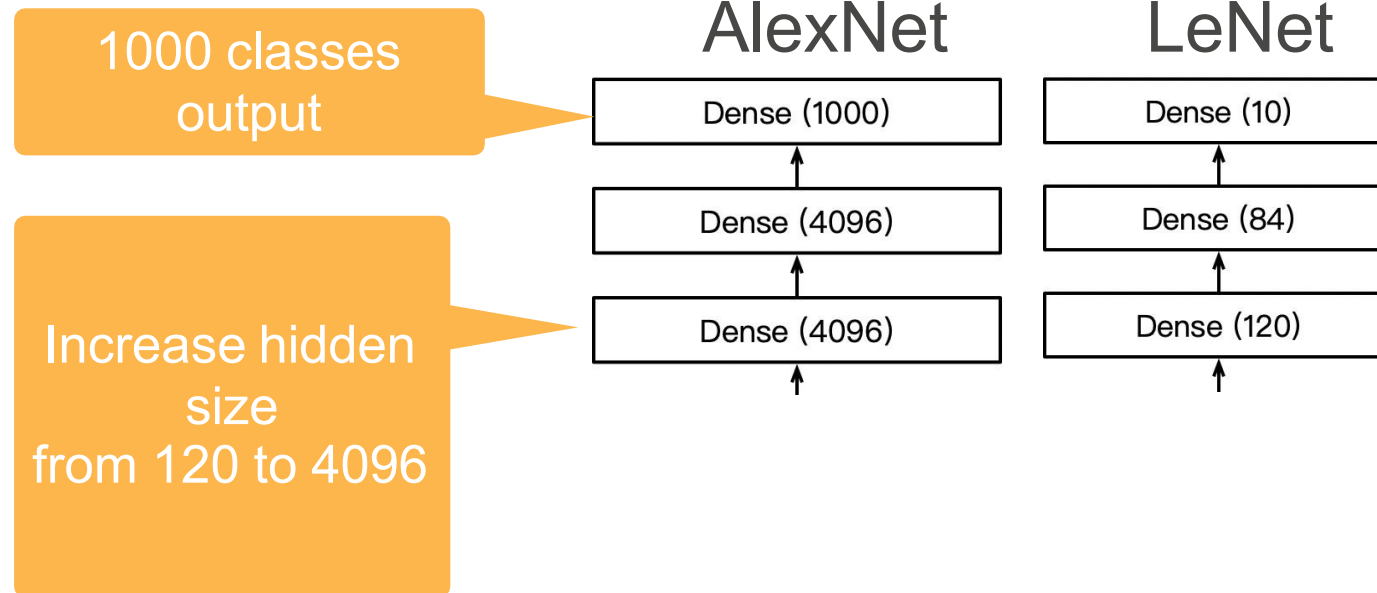
Larger kernel size,
stride because of the
increased image size,
and more output
channels.



AlexNet Architecture



AlexNet Architecture



Comparing CNN, AlexNet, and ResNet in Keras



- Evolution of Depth, Optimization, and Transfer Learning??

Simple CNN



- - Layers: 2 conv + pooling + 2 dense
- - Params: $\sim 0.2M$
- - Performance: $\sim 60\%$ on CIFAR-10
- - Strength: Fast, simple
- - Limitation: Underfits complex data

AlexNet (2012)



- - 5 conv + 3 dense layers (~60M params originally)
- - Introduced ReLU, dropout, augmentation
- - Deeper than CNN but unstable without normalization

ResNet (2015)



- - 50+ layers with skip connections ($y = F(x) + x$)
- - Solves vanishing gradient & degradation
- - Enables very deep, trainable networks

Code Recap



- - Dataset: CIFAR-10
- - Models: Simple CNN, AlexNet-like, ResNet50
- - Same optimizer and preprocessing
- - Metric: Validation accuracy/loss

Training Results: Accuracy



- Interpretation:
 - - Simple CNN: Low, stable
 - - AlexNet-like: Improves but overfits
 - - ResNet50: High, stable — best generalization

Training Results: Loss



- Interpretation:
- - Simple CNN: Underfits (high bias)
- - AlexNet-like: Overfits (high variance)
- - ResNet: Balanced

Vanishing Gradients



- Gradients shrink as they propagate backward.
- → Early layers stop learning.
- → Deeper networks may degrade without residuals.

Skip Connections



- Equation: $y = F(x) + x$
- Backprop: $\partial L / \partial x = \partial L / \partial y * (1 + \partial F / \partial x)$
- Ensures gradient always has a direct path → **stable training.**

Transfer Learning



- - Uses pretrained weights (ImageNet → CIFAR)
- - Benefits: Faster, more accurate, less overfitting
- - **Freeze base**, train small dense head

Instead of training a deep neural network from scratch—which requires millions of labeled images and extensive computation—you can **leverage a pretrained model** (e.g., ResNet50 trained on ImageNet) and **transfer its learned features** to a new, smaller dataset (e.g., classifying medical images, plant species, or custom objects).



Step-by-Step Explanation

1. Pretrained Model (Source Domain)

1. Use a deep CNN like **ResNet50**, which has already learned to recognize general image features (edges, shapes, textures) from **ImageNet** (over 1 million labeled images across 1000 classes).

2. Target Domain

1. Suppose you have a dataset of **X-ray chest images** for classifying **Normal vs Pneumonia** (only a few thousand samples).
2. Training a deep CNN from scratch would cause **overfitting** and **require huge resources**.

3. Transfer Learning Strategy

1. **Load pretrained ResNet50** with weights from ImageNet.
2. **Remove the top (fully connected) layer**, which outputs 1000 ImageNet classes.
3. **Freeze the convolutional base layers** to retain learned low-level features.
4. **Add new fully connected layers** suitable for your target classes (e.g., **2 neurons for binary classification**).
5. **Train only the new top layers**, or fine-tune a few upper convolutional layers.



```
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.optimizers import Adam

# Load pretrained ResNet50 model + higher Level Layers
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Freeze convolutional base
for layer in base_model.layers:
    layer.trainable = False

# Add custom classifier layers
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
predictions = Dense(2, activation='softmax')(x)

# Combine base + new Layers
model = Model(inputs=base_model.input, outputs=predictions)
```



```
# Add custom classifier layers
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
predictions = Dense(2, activation='softmax')(x)

# Combine base + new layers
model = Model(inputs=base_model.input, outputs=predictions)

# Compile
model.compile(optimizer=Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train on new dataset
train_datagen = ImageDataGenerator(rescale=1./255)
train_generator = train_datagen.flow_from_directory('data/train', target_size=(224, 224))
val_generator = train_datagen.flow_from_directory('data/val', target_size=(224, 224))

model.fit(train_generator, validation_data=val_generator, epochs=10)
```

Overfitting & Overparameterization



- Simple CNN: Underfits (~ 0.2 M params)
- AlexNet-like: Overfits (~ 10 M)
- ResNet50: Generalizes (~ 25 M, pretrained)

Keras example



```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

# --- Load and preprocess CIFAR-10 ---
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0
y_train, y_test = to_categorical(y_train, 10), to_categorical(y_test, 10)
```



```
# --- Define Models ---
```

```
def simple_cnn():  
    model = models.Sequential([  
        layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),  
        layers.MaxPooling2D((2,2)),  
        layers.Conv2D(64, (3,3), activation='relu'),  
        layers.MaxPooling2D((2,2)),  
        layers.Flatten(),  
        layers.Dense(64, activation='relu'),  
        layers.Dense(10, activation='softmax')  
    ])  
    return model
```



```
def alexnet_like():
    model = models.Sequential([
        layers.Conv2D(64, (3,3), activation='relu', padding='same', input_shape=(32,32,3)),
        layers.MaxPooling2D((2,2)),
        layers.Conv2D(128, (3,3), activation='relu', padding='same'),
        layers.MaxPooling2D((2,2)),
        layers.Conv2D(256, (3,3), activation='relu', padding='same'),
        layers.Conv2D(256, (3,3), activation='relu', padding='same'),
        layers.Conv2D(128, (3,3), activation='relu', padding='same'),
        layers.MaxPooling2D((2,2)), # Final pooling layer
        layers.Flatten(),
        layers.Dense(512, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(10, activation='softmax')
    ])
    return model
```



```
def resnet_model():
    base = ResNet50(weights='imagenet', include_top=False, input_shape=(32,32,3))
    base.trainable = False # Freeze pretrained layers
    model = models.Sequential([
        base,
        layers.GlobalAveragePooling2D(),
        layers.Dense(256, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    return model
```



```
# --- Prepare models ---
models_dict = {
    "Simple CNN": simple_cnn(),
    "AlexNet-like": alexnet_like(),
    "ResNet50": resnet_model()
}
```

```
history_dict = {}
results = {}
```

```
# --- Train & evaluate ---
for name, model in models_dict.items():
    print(f"\n🌱 Training {name}...")
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    hist = model.fit(x_train, y_train,
                     epochs=3, batch_size=64, validation_split=0.1, verbose=1)
    history_dict[name] = hist
    loss, acc = model.evaluate(x_test, y_test, verbose=0)
    results[name] = acc
    print(f"{name} Test Accuracy: {acc:.3f}")
```



```
# Accuracy
plt.subplot(1,2,1)
for name, hist in history_dict.items():
    plt.plot(hist.history['val_accuracy'], label=f"{name}")
plt.title('Validation Accuracy Comparison')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Loss
plt.subplot(1,2,2)
for name, hist in history_dict.items():
    plt.plot(hist.history['val_loss'], label=f"{name}")
plt.title('Validation Loss Comparison')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()

# --- Final Comparison ---
print("\n📊 Final Test Accuracy Comparison:")
for name, acc in results.items():
    print(f"{name:<15}: {acc:.3f}")
```



✿ Training Simple CNN...

Epoch 1/3

704/704 ————— 37s 51ms/step - accuracy: 0.3566 - loss: 1.7718 - val_accuracy: 0.5584 - val_loss: 1.2492

Epoch 2/3

704/704 ————— 35s 49ms/step - accuracy: 0.5724 - loss: 1.2260 - val_accuracy: 0.6190 - val_loss: 1.0871

Epoch 3/3

704/704 ————— 37s 52ms/step - accuracy: 0.6241 - loss: 1.0674 - val_accuracy: 0.6348 - val_loss: 1.0288

Simple CNN Test Accuracy: 0.642

✿ Training AlexNet-like...

Epoch 1/3

704/704 ————— 533s 754ms/step - accuracy: 0.2282 - loss: 2.0182 - val_accuracy: 0.4994 - val_loss: 1.3489

Epoch 2/3

704/704 ————— 530s 708ms/step - accuracy: 0.5168 - loss: 1.3389 - val_accuracy: 0.5988 - val_loss: 1.1259

Epoch 3/3

704/704 ————— 490s 696ms/step - accuracy: 0.6064 - loss: 1.1180 - val_accuracy: 0.6508 - val_loss: 0.9945

AlexNet-like Test Accuracy: 0.638

✿ Training ResNet50...

Epoch 1/3

704/704 ————— 166s 227ms/step - accuracy: 0.2005 - loss: 2.2054 - val_accuracy: 0.3204 - val_loss: 1.9553

Epoch 2/3

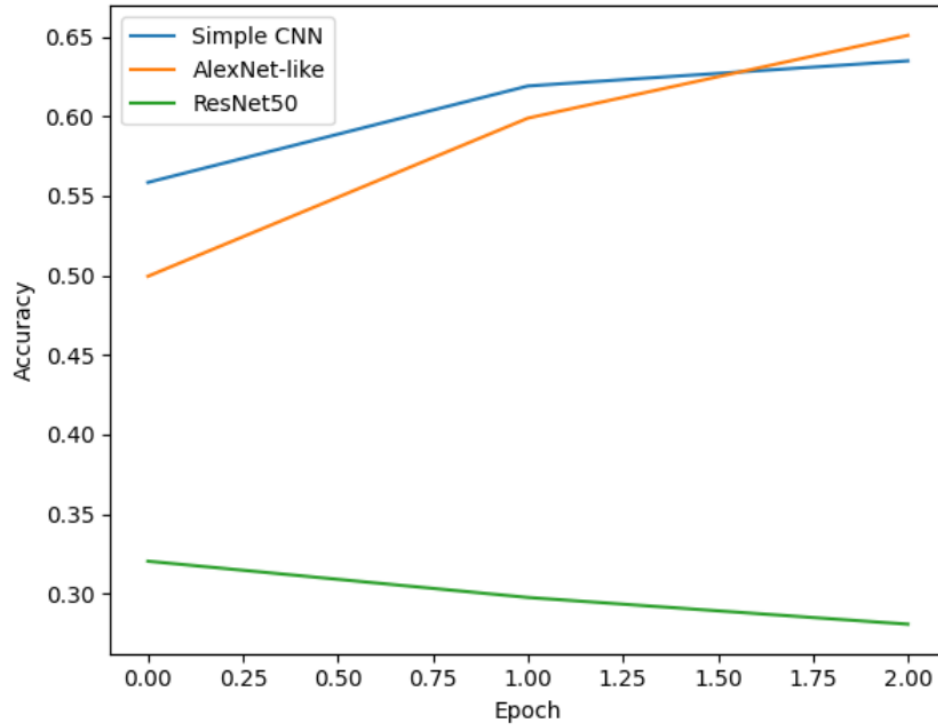
704/704 ————— 189s 209ms/step - accuracy: 0.2998 - loss: 1.9183 - val_accuracy: 0.2976 - val_loss: 1.9379

Epoch 3/3

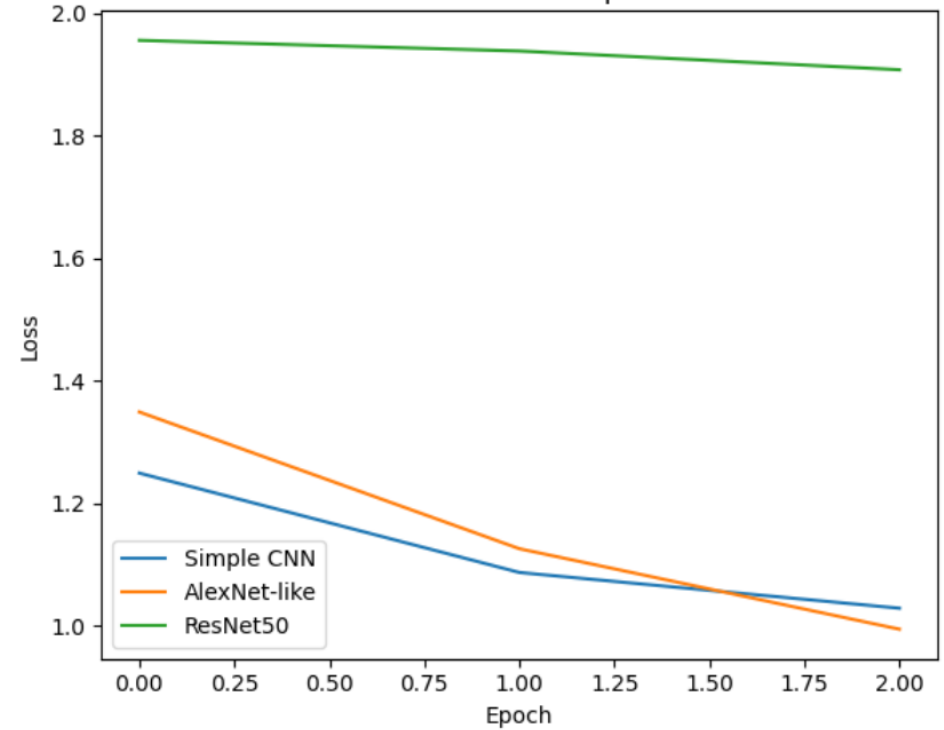
704/704 ————— 136s 194ms/step - accuracy: 0.3226 - loss: 1.8625 - val_accuracy: 0.2808 - val_loss: 1.9073

ResNet50 Test Accuracy: 0.285

Validation Accuracy Comparison



Validation Loss Comparison



Final Test Accuracy Comparison:

Simple CNN : 0.642
 AlexNet-like : 0.638
 ResNet50 : 0.285



Model	Depth	Parameters	Expected Accuracy (1 Epoch)	Notes
Simple CNN	Shallow (2 conv blocks)	~0.2M	~60%	Fast, underfits
AlexNet-like	Deep (5 conv + 2 FC)	~10M	~70%	Larger receptive fields
ResNet50	50 layers	~25M	~80%	Skip connections help generalization

1. Vanishing Gradients and Deep Networks



- As neural networks become deeper (many stacked layers), the **gradients** used during backpropagation tend to become **very small** in the early layers. When this happens, earlier layers **learn very slowly or not at all** — this is known as the **vanishing gradient**
- Simple CNN: Only 2–3 convolutional layers — shallow, so gradients flow easily.
- AlexNet-like: Deeper (5 conv + 2 dense) — prone to vanishing gradients if not designed carefully.
- ResNet50: Introduces skip (residual) connections, which bypass one or more layers and allow gradients to flow directly back to earlier layers.

2. Transfer Learning (ResNet50)

- Transfer learning uses **weights pre-trained on a large dataset (like ImageNet)** and applies them to a **smaller or different dataset (like CIFAR-10)**.
The model already knows general visual patterns (edges, shapes, textures), so it only needs to adapt to new classes.
- The ResNet50 base is pre-trained on ImageNet. You freeze its convolutional layers (so they're not retrained). You add a small dense head to classify CIFAR-10.
- ♦ Benefits:
 - Faster convergence (learns in fewer epochs).
 - Higher accuracy with limited data.
 - Less overfitting, since the convolutional base is already generalized.

3. Overparameterization and Overfitting



- More parameters (weights) mean more capacity to learn complex patterns, but also a higher risk of memorizing the training data — that's overfitting.
- Simple CNN: Small parameter count ($\sim 200K$) — underfits complex data.
- AlexNet-like: Large ($\sim 10M$) — may overfit small datasets like CIFAR-10.
- ResNet50: Very large ($\sim 25M$) but benefits from pretraining and regularization.

4. Skip Connections and Model Depth



- Deeper networks can theoretically represent more complex functions. However, before ResNet, adding layers degraded performance because optimization became unstable.
- **ResNet's Innovation**: Residual learning allows 50, 101, or even 152 layers to be trained effectively. The residual path ensures that even if deeper layers fail to learn something useful, the model still performs as if it were shallower — avoiding degradation.

5. Efficiency Trade-offs



Model	Accuracy	Training Speed	Memory Use	Discussion
Simple CNN	Low (~60%)	Very fast	Low	Good baseline; underfits
AlexNet-like	Medium (~70%)	Moderate	High	Heavy, overfits small data
ResNet50 (Transfer)	High (~80–85%)	Fast (frozen base)	Medium	Most efficient in accuracy/time ratio

Should we always use the deepest model? → Not necessarily. For small datasets, a simpler CNN might generalize better.

Is transfer learning always better? → Only if the source and target domains are related. (e.g., ImageNet → CIFAR works, but ImageNet → Medical MRI might not.)

6. Interpreting Training Curves



- Show validation accuracy/loss plots:
- Simple CNN: slow improvement, low ceiling.
 - high bias (underfits)
- AlexNet-like: high training accuracy but validation plateaus — overfitting.
 - high variance (overfits)
- ResNet50: high, stable validation accuracy — good generalization.
 - balanced (good bias–variance trade-off)

Why doesn't adding more layers always increase performance?



- Because deeper networks are harder to optimize and can suffer from vanishing/exploding gradients, degradation, or overfitting.
 - Gradients vanish as they propagate backward, especially through many non-linear activations.
 - Deeper networks can degrade, meaning training accuracy itself worsens — not because of overfitting, but because the optimization landscape becomes unstable.
 - Overfitting can also occur when the model memorizes training data but fails to generalize.

What happens if we unfreeze all layers of ResNet50 on CIFAR-10?

- The model will likely overfit and train slowly, possibly with little in

Correct Strategy:
Freeze the convolutional base (feature extractor) initially.

Unfreeze the top few blocks only if you have enough data or want fine-tuning.

es) — its
l images.

- If you unfreeze all layers:
 - You allow the model to update the pre-trained weights, but since CIFAR-10 is small, it doesn't have enough data to re-learn good general features.
 - The optimizer will disturb the well-learned low-level filters, leading to catastrophic forgetting.
 - Training time and memory usage increase significantly.

How do skip connections improve gradient flow mathematically?



- They allow gradients to bypass nonlinear transformations, ensuring that the gradient can flow directly to earlier layers.
- Use an alternate route:
 - preserves gradient magnitude during backpropagation.
 - Enables stable training of very deep networks (e.g., 50–150+ layers).
 - Prevents degradation — deeper models perform at least as well as shallower ones.

Would AlexNet perform better with Batch Normalization?



- Yes — adding Batch Normalization (BN) layers would likely improve convergence, stability, and accuracy.
- BN after each convolutional layer: :
 - Stabilizes gradient flow.(Training becomes smoother and faster.)
 - Allows higher learning rates.(Test accuracy improves (especially on small datasets like CIFAR-10).
 - Acts as a regularizer (reduces overfitting)=> Dropout can often be reduced because BN provides implicit regularization.

When might transfer learning not be helpful?



- When the source domain and target domain are too different — the learned representations are not useful.
le **Solution:**
- **Use self-supervised pretraining (e.g., SimCLR, MAE) on unlabeled data from your target domain.**
- **Train from scratch if the dataset is large enough.** (e.g., ImageNet, COCO, etc.).
- Later layers capture domain-specific features (animal faces, textures, etc.).
- If domains differ drastically, these learned representations are not useful, and might even hurt performance (negative transfer).

Summary



Question

Why doesn't more depth always help?

What if we unfreeze all ResNet layers?

How do skip connections help?

AlexNet + BN?

When not to use transfer learning?

Key Concept

Optimization issues

Fine-tuning & overfitting

Gradient flow

Normalization

Domain mismatch

Core Answer

Deep networks may degrade due to vanishing gradients; ResNet fixes it.

Overfits small data; better to freeze or partially fine-tune.

Preserve identity gradient; prevent vanishing.

Yes — improves stability and accuracy.

When pretraining domain differs from target domain.

Questions



- 1. Why doesn't adding layers always help?
Deep networks degrade w/o skip connections
- 2. What if we unfreeze all ResNet layers?
Overfits small data
- 3. How do skip connections help?
Skip connections preserve gradient flow
- 4. Would AlexNet improve with BatchNorm?
BN stabilizes & regularizes
- 5. When might transfer learning fail?
Fails with domain mismatch

More Tricks



- Change activation function from sigmoid to ReLu (no more vanishing gradient)
- Add a dropout layer after two hidden FFN layers (better robustness / regularization)
- Data augmentation

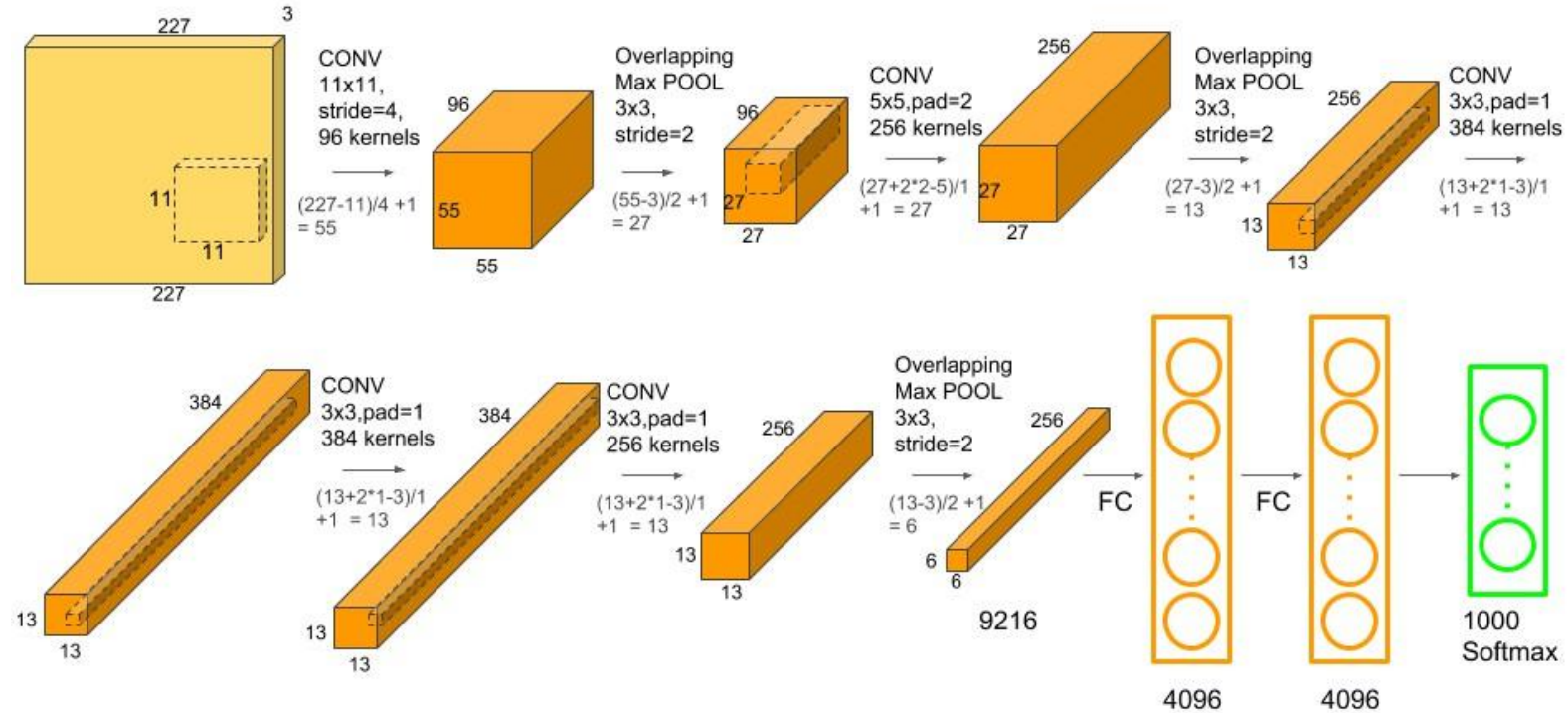
Data Augmentation



- Create additional training data with existing data



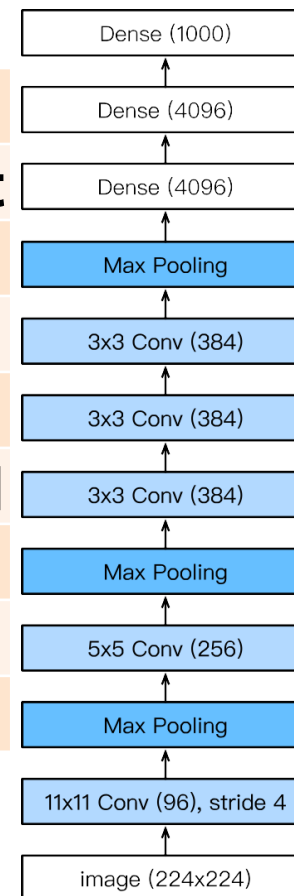
AlexNet



Complexity



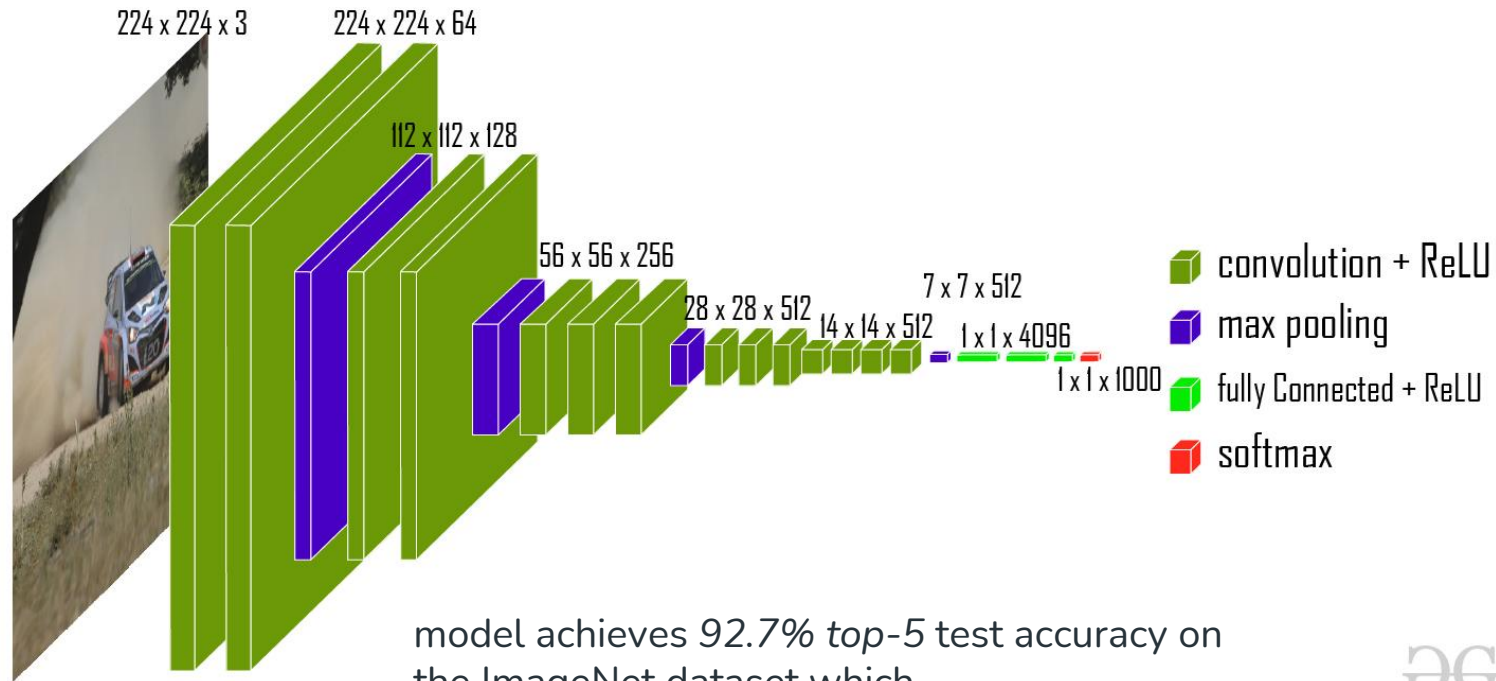
	#parameters		FLOP	
	AlexNet	LeNet	AlexNet	LeNet
Conv1	35K	150	101M	1.2M
Conv2	614K	2.4K	415M	2.4M
Conv3-5	3M		445M	
Dense1	26M	0.48M	26M	0.48M
Dense2	16M	0.1M	16M	0.1M
Total	46M	0.6M	1G	4M
Increase	11x	1x	250x	1x



FLOPS, or floating-point operations per second, is a metric for computer performance that quantifies the number of floating-point calculations a system can perform in one second.

VGG

Visual Geometry Group



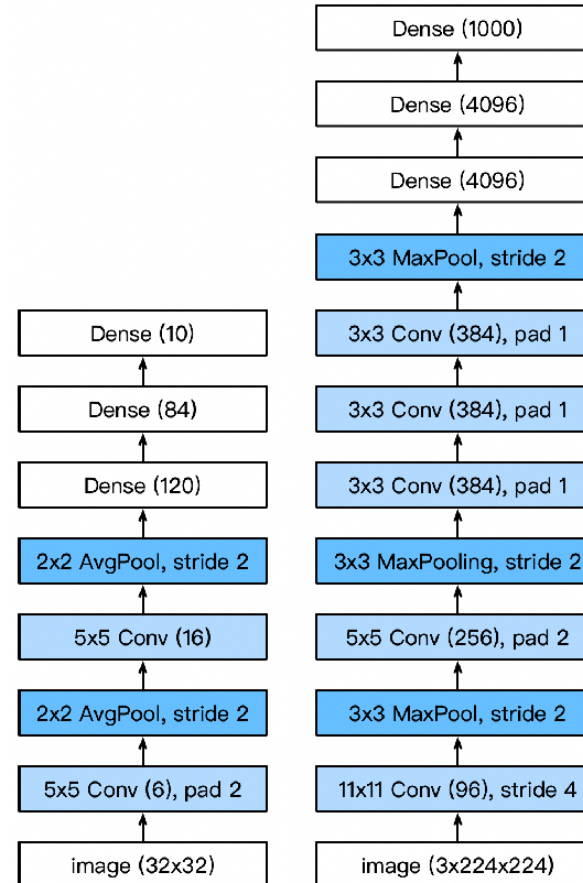
model achieves 92.7% *top-5* test accuracy on the ImageNet dataset which contains 14 million images belonging to 1000 classes



VGG



- AlexNet is deeper and bigger than LeNet to get performance
- Go even bigger & deeper?
- Options
 - More dense layers (too expensive)
 - **More** convolutions
 - Group into **blocks**

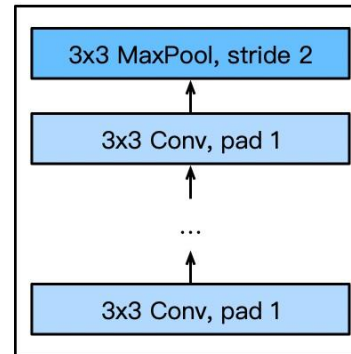


VGG Blocks

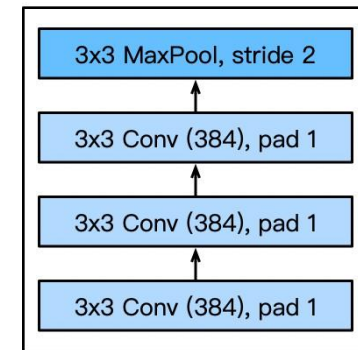


- Deeper vs. wider?
 - 5x5 convolutions
 - 3x3 convolutions (more)
 - **Deep & narrow better**
- VGG block
 - 3x3 convolutions (pad 1) (**n layers, m channels**)
 - 2x2 max-pooling (stride 2)

VGG block

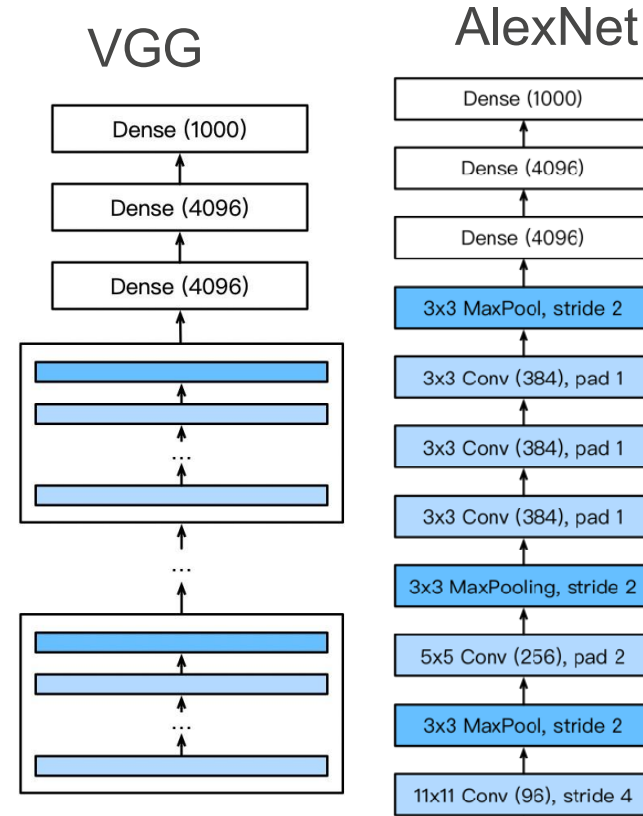


Part of AlexNet



VGG Architecture

- Multiple VGG blocks followed by dense layers
- Vary the repeating number to get different architectures, such as VGG-16, VGG-19, ...



Going Deeper



- LeNet (1995)
 - 2 convolution + pooling layers
 - 2 hidden dense layers
- AlexNet
 - Bigger and deeper LeNet
 - ReLu, Dropout, preprocessing
- VGG
 - Bigger and deeper AlexNet (repeated VGG blocks)

Residual Networks

A pre-trained Convolutional Neural Network
for image classification and computer vision.

Best paper CVPR 2016

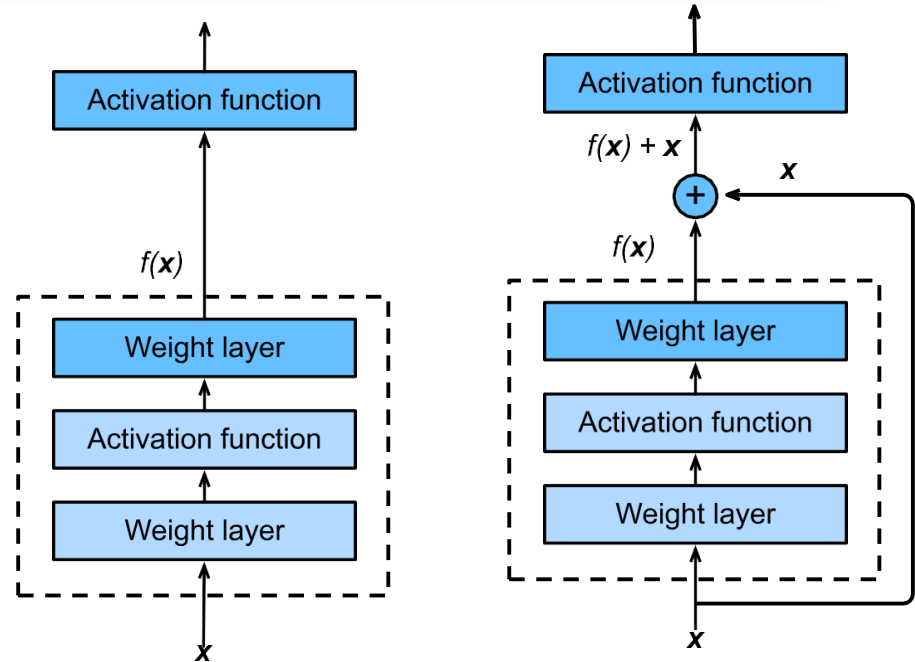
Types of ResNets:

1. ResNet18, 2. ResNet34, 3. ResNet50, 4. ResNet101, 5. ResNet152

Suffix number refers to the number of layers used in their network architecture.

Residual Networks

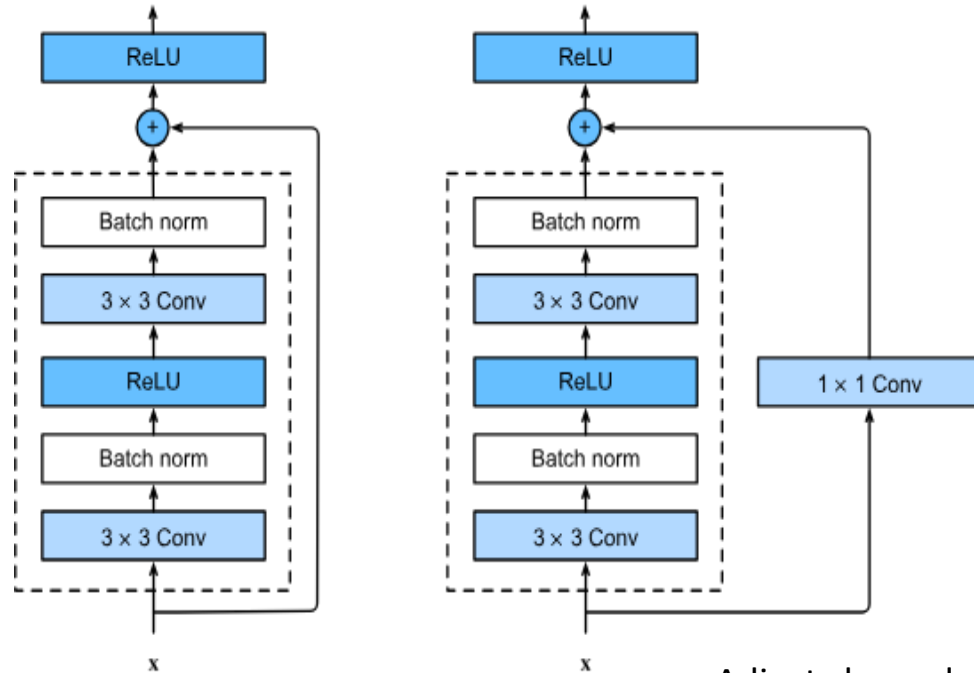
- Adding a layer **changes** function class
- We want to **add to** the function class
- 'Taylor expansion' style $f(x) = x + g(x)$ parametrization



Regular block the portion within the dotted-line box must directly learn the mapping

Residual block, the portion within the dotted-line box needs to learn the residual mapping $g(x)=f(x)-x$, making the identity mapping $f(x)=x$ easier to learn.

Residual Networks



This block includes a convolutional layer with a stride of (2, 2) to downsample the input, and a convolutional layer with a (1, 1) filter size to change the depth of the input to match the output.

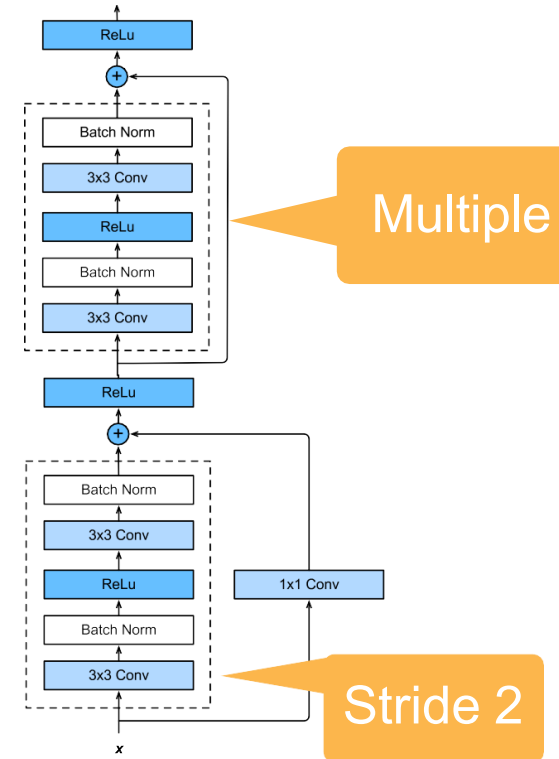
Add the input to the output before applying the ReLU

Adjust channels and resolution by means of a 1x1 convolution before adding

1x1 convolutional layer to transform the input into the desired shape for the addition operation, if the number of channels has changed

ResNet Module

- Downsample per module (stride=2)
- Enforce some nontrivial nonlinearity per module (via 1x1 convolution)
- Stack up in blocks



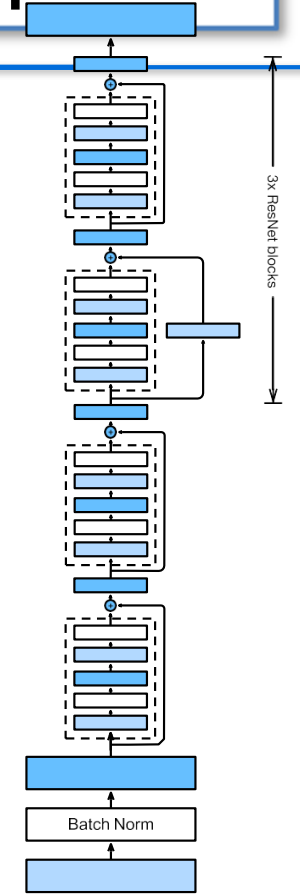
```
blk = nn.Sequential()
for i in range(num_residuals):
    if i == 0 and not first_block:
        blk.add(Residual(num_channels,
                          use_1x1conv=True, strides=2))
    else:
        blk.add(Residual(num_channels))
```

Putting it all together



- Same block structure as e.g. VGG or GoogleNet
- Residual connection to add to expressiveness
- Pooling/stride for dimensionality reduction
- Batch Normalization for capacity control

... train it at scale ...



ResNext



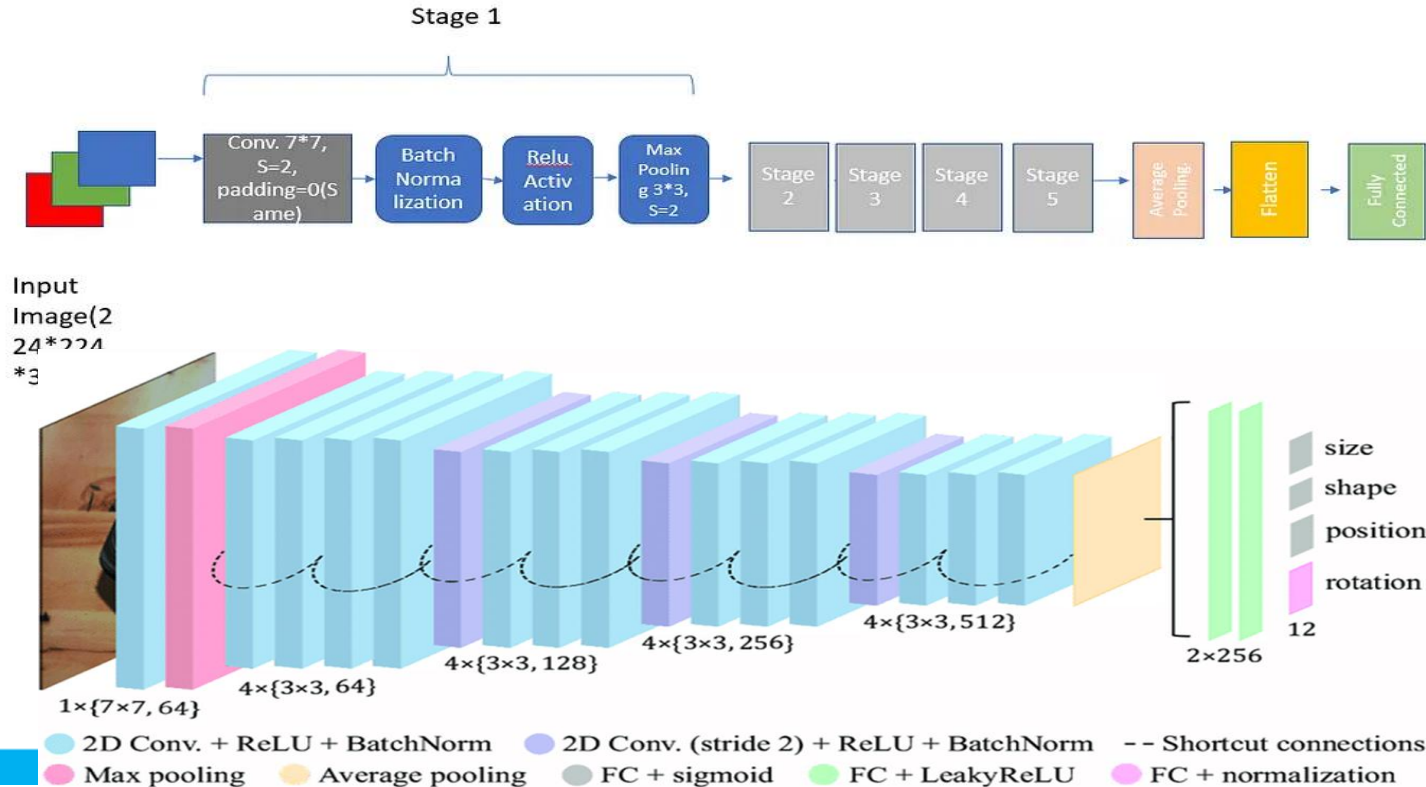
RexNext budget

- Slice blocks into 32 sub-blocks
- Can use more dimensions
- Higher accuracy

stage	output	ResNet-50	ResNeXt-50 (32×4d)
conv1	112×112	7×7, 64, stride 2	7×7, 64, stride 2
conv2	56×56	3×3 max pool, stride 2	3×3 max pool, stride 2
		$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128, C=32 \\ 1\times 1, 256 \end{bmatrix} \times 3$
conv3	28×28	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256, C=32 \\ 1\times 1, 512 \end{bmatrix} \times 4$
conv4	14×14	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512, C=32 \\ 1\times 1, 1024 \end{bmatrix} \times 6$
conv5	7×7	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 1024 \\ 3\times 3, 1024, C=32 \\ 1\times 1, 2048 \end{bmatrix} \times 3$
	1×1	global average pool 1000-d fc, softmax	global average pool 1000-d fc, softmax
# params.		25.5×10^6	25.0×10^6
FLOPs		4.1×10^9	4.2×10^9

ResNet50 consists of 16 residual blocks, with each block consisting of several convolutional layers with residual connections.

The architecture also includes pooling layers, fully connected layers, and a softmax output layer for classification.



Stage 1:

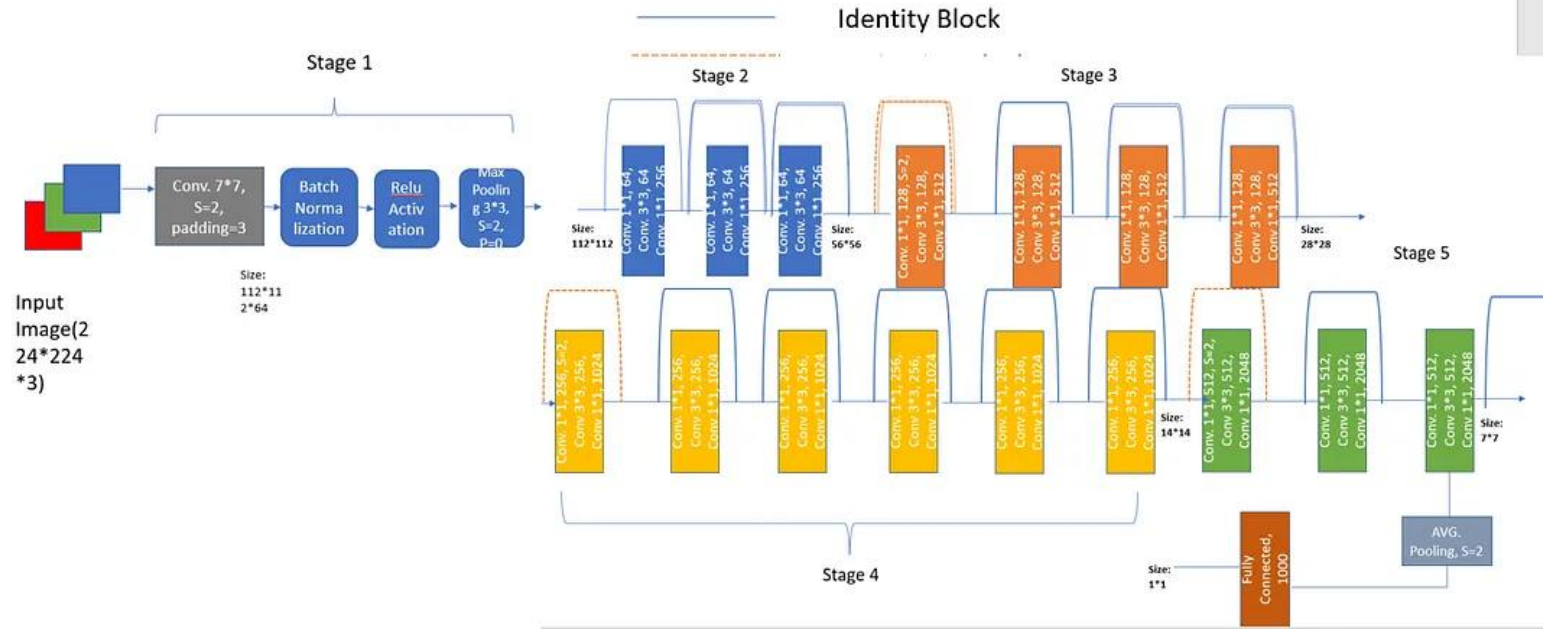
First convolutional layer, which takes as input the raw image with dimensions $(224, 224, 3)$ and outputs a feature map with dimensions $(112, 112, 64)$.

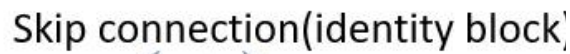
This layer uses 64 filters, each with a kernel size of $(7, 7)$ and a stride of $(2, 2)$ to downsample the input image by a factor of 2 in both the width and height dimensions.

After the convolutional layer, a batch normalization layer is applied to normalize the activations, followed by a ReLU activation function to introduce nonlinearity.

Finally, a max pooling layer with a pool size of $(3, 3)$ and a stride of $(2, 2)$ is applied to further downsample the feature map by a factor of 2 in both dimensions.

This produces a feature map with dimensions (56, 56, 64), which serves as the input to the subsequent convolutional blocks in the ResNet50 architecture.

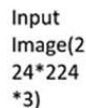




The output of the first stage is forwarded to the first layer of convolution with 64 kernel filters and a size of 1×1 o/p of this feed to 3×3 convolution size with the same 64 kernel filters, o/p this feed to conv. 1×1 with 256 kernel filters.

In this uses 9 Layers of convolutional layer followed by batch normalization and relu activation.

The identity block(connecting blue color lines from one to another block of convolutions).

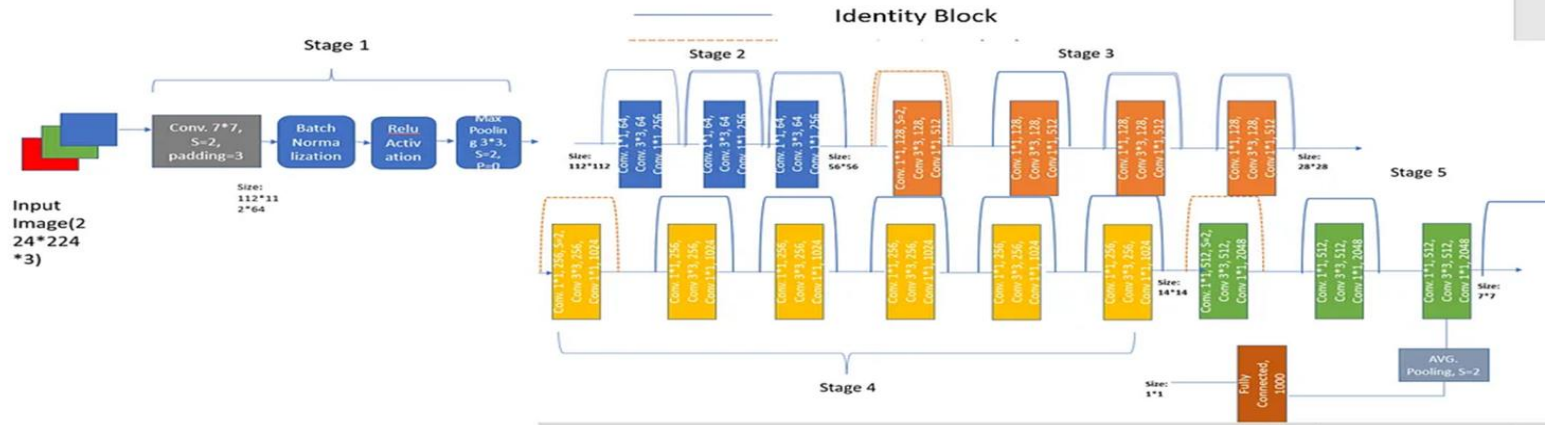
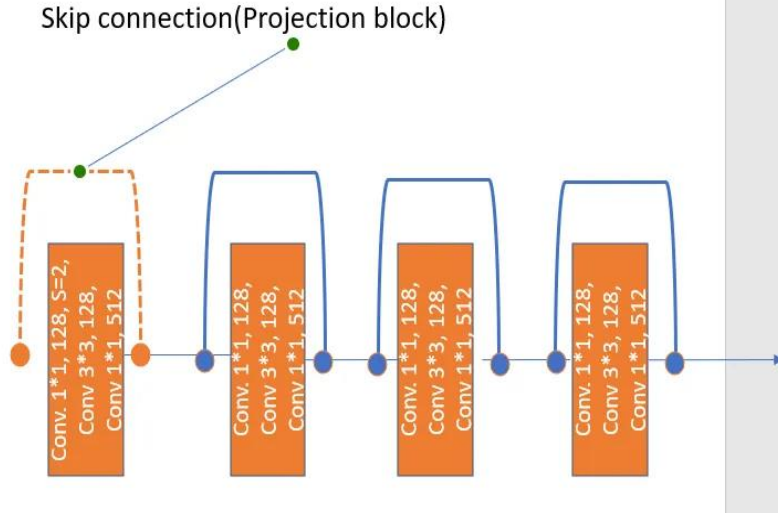


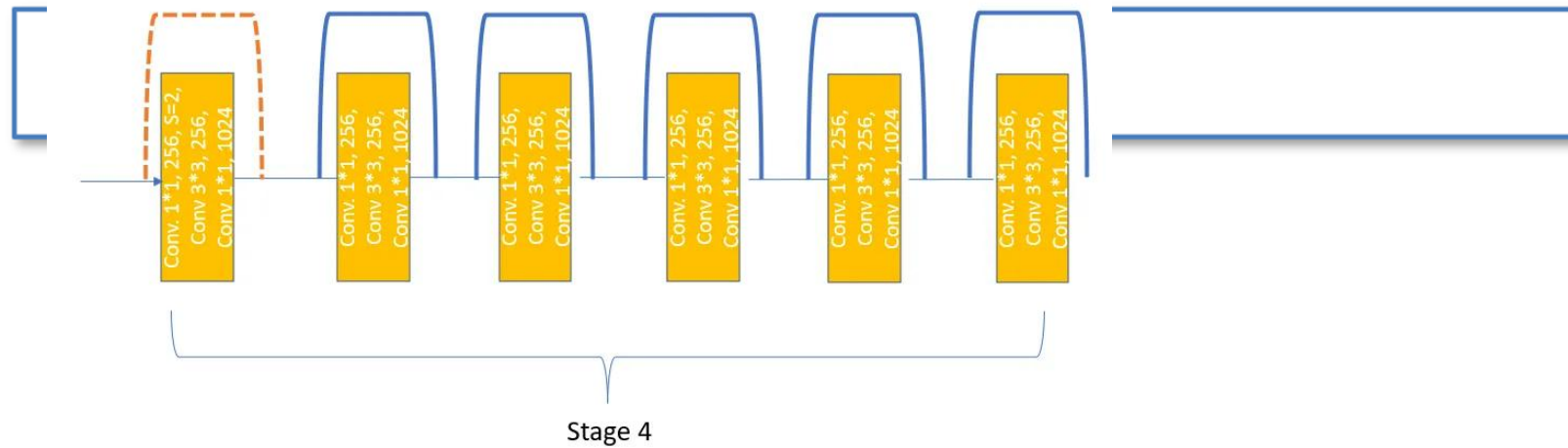
Stage 3:

In this, 12 layers of convolutional layer followed by batch normalization and relu activation are used.

Here Projection block used as the dimension of input and output is not the same, bypassing the first three layers convolutional layer, in the first layer convoluting with the size of kernel 1×1 with 128 filters (stride=2) to downsample the input and to change the depth of the input to match the output followed by convolution of 3×3 , with, a same number of filters and conv.

1×1 , kernel filters=512. Further use 9 layers of convolution with 1×1 , filters=128 $\Rightarrow$ 3×3 , filters=128 $\Rightarrow$ 1×1 , filters=512. **The output of these 12th layers with size of image 28×28 forwards to relu activation before stage 4.**





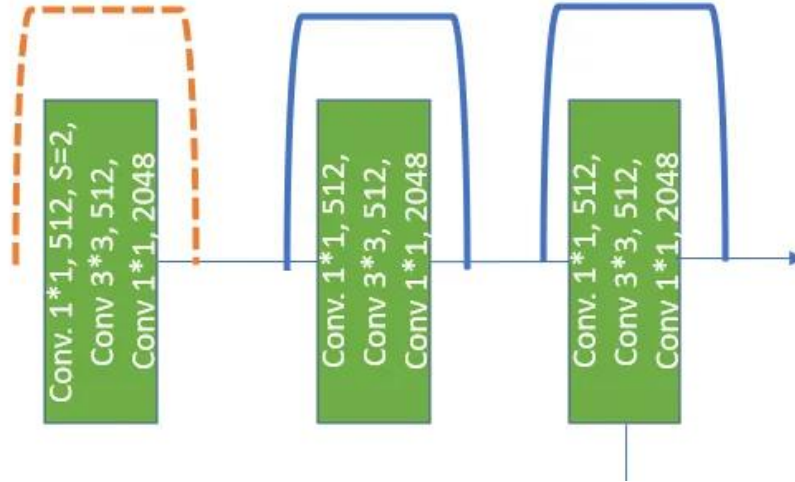
Stage4:

In this stage uses 18 layers of convolution each followed by batch normalization and relu activation function.

Output from relu activation has been forwarded to the first layer of the fourth stage where the dimension of image size is different **so use projection block here in which input is forwarded to the convolution block to match the size of i/p with o/p at the second conv. block.**

The rest of the process remains the same as like previous stage convoluting layer by layer to understand more deep features of the image.

Stage 5

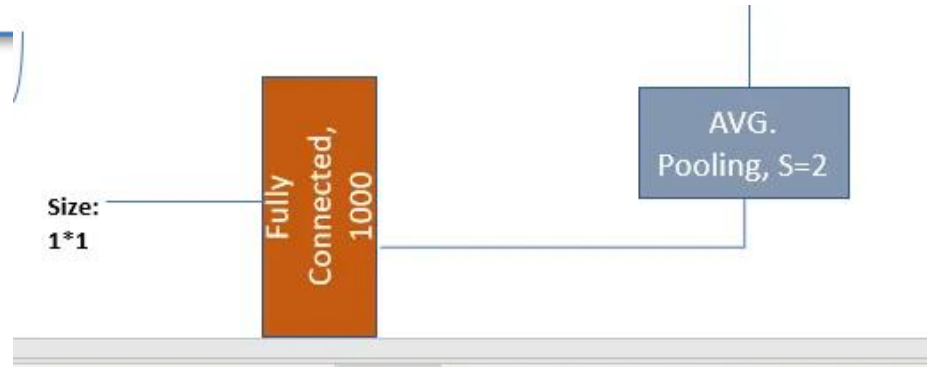


Stage 5:

In this stage uses 9 layers of convolution each followed by batch normalization and relu activation function.

Output from relu activation has been forwarded to with size of image 14*14 the first layer of the 5th stage where the dimension of image size is different so use projection block here in which input is forwarded to the convolution block to match the size of i/p with o/p at the second conv. block.

The very first layer of the first convolutional block reducing the size of the image using 512 kernel filters with size of 1*1(stride=2) followed by conv 3*3 with 512 filters and conv. 1*1 with 2048 filters.



Finally, the output of the last relu activation unit at the fifth stage will be passed on to **Average Pooling** which reduces the spatial dimensions of the output tensor to a **vector** by applying global average pooling with stride size=2.

This operation computes the average of each feature map, resulting in a feature vector with the same number of channels as the number of filters in the last convolutional layer.

Defining convolutional block function



```
from tensorflow.keras.layers import Input, Conv2D, BatchNormalization,
Activation, MaxPooling2D, Dense, GlobalAveragePooling2D, Add
from tensorflow.keras import Model

def conv_block(x, filters, kernel_size, strides, padding='same'):
    x = Conv2D(filters=filters, kernel_size=kernel_size, strides=strides,
padding=padding)(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    return x
```



Identity block and Projection block function:

```
def identity_block(x, filters):  
    shortcut = x  
    x = conv_block(x, filters=filters, kernel_size=(1, 1), strides=(1, 1))  
    x = conv_block(x, filters=filters, kernel_size=(3, 3), strides=(1, 1))  
    x = Conv2D(filters=filters * 4, kernel_size=(1, 1))(x)  
    x = BatchNormalization()(x)  
    x = Add()([x, shortcut])  
    x = Activation('relu')(x)  
    return x
```

identity block used when the dimension of output and input are same

```
def projection_block(x, filters, strides):  
    shortcut = x  
    x = conv_block(x, filters=filters, kernel_size=(1, 1), strides=strides)  
    x = conv_block(x, filters=filters, kernel_size=(3, 3), strides=(1, 1))  
    x = Conv2D(filters=filters * 4, kernel_size=(1, 1))(x)  
    x = BatchNormalization()(x)  
    shortcut = Conv2D(filters=filters * 4, kernel_size=(1, 1),  
strides=strides)(shortcut)  
    shortcut = BatchNormalization()(shortcut)  
    x = Add()([x, shortcut])  
    x = Activation('relu')(x)  
    return x
```



```
def ResNet50(input_shape=(224, 224, 3), num_classes=1000):  
    inputs = Input(shape=input_shape)  
  
    # initial conv layer  
    x = conv_block(inputs, filters=64, kernel_size=(7, 7),  
strides=(2, 2), padding='')  
    x = MaxPooling2D(pool_size=(3, 3), strides=(2, 2),  
padding='same')(x)  
  
    # conv block 1  
    x = projection_block(x, filters=64, strides=(1, 1))  
    x = identity_block(x, filters=64)  
    x = identity_block(x, filters=64)  
  
    # conv block 2  
    x = projection_block(x, filters=128, strides=(2, 2))  
    x = identity_block(x, filters=128)  
    x = identity_block(x, filters=128)  
    x = identity_block(x, filters=128)
```



```
# conv block 3
x = projection_block(x, filters=256, strides=(2, 2))
x = identity_block(x, filters=256)
x = identity_block(x, filters=256)
x = identity_block(x, filters=256)
x = identity_block(x, filters=256)
x = identity_block(x, filters=256)

# conv block 4
x = projection_block(x, filters=512, strides=(2, 2))
x = identity_block(x, filters=512)
x = identity_block(x, filters=512)

# global average pooling and dense layer
x = GlobalAveragePooling2D()(x)
outputs = Dense(num_classes, activation='softmax')(x)

model = Model(inputs=inputs, outputs=outputs)
return model
```

ResNet



In Python



```
[1] import matplotlib.pyplot as plt
import numpy as np
import os
import PIL
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.python.keras.layers import Dense, Flatten
from tensorflow.keras.models import Sequential
```

✓ Preparing The Data

```
[2] import pathlib
flowers_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz"
flowers_data = tf.keras.utils.get_file('flower_photos', origin=flowers_url, untar=True)
flowers_data = pathlib.Path(flowers_data)
```

📄 Downloading data from https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz
228813984/228813984 ————— 5s 0us/step

```
[3] print(flowers_data)
```

📄 /root/.keras/datasets/flower_photos

```
[4] all_sunflowers = list(flowers_data.glob('sunflowers/*'))
    print(all_sunflowers[1])
    PIL.Image.open(str(all_sunflowers[1]))
```

→ /root/.keras/datasets/flower_photos/sunflowers/13648603305_1268eda8b7_n.jpg



```
[5] height,width=180,180

    training_batch_size=32

    train_set = tf.keras.preprocessing.image_dataset_from_directory(
        flowers_data,
        validation_split=0.2,
        subset="training",
        seed=123,
        image_size=(height,width),
        batch_size=training_batch_size)
```



✓
3s

```
[5] height,width=180,180
```

```
training_batch_size=32
```

```
train_set = tf.keras.preprocessing.image_dataset_from_directory(  
    flowers_data,  
    validation_split=0.2,  
    subset="training",  
    seed=123,  
    image_size=(height,width),  
    batch_size=training_batch_size)
```



```
Found 3670 files belonging to 5 classes.  
Using 2936 files for training.
```

✓
0s

```
[6] validation_set = tf.keras.preprocessing.image_dataset_from_directory(  
    flowers_data,  
    validation_split=0.2,  
    subset="validation",  
    seed=123,  
    image_size=(height, width),  
    batch_size=training_batch_size)
```



```
Found 3670 files belonging to 5 classes.  
Using 734 files for validation.
```

✓
0s

```
[7] image_cat = train_set.class_names  
    print(image_cat)
```



```
['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']
```

Training The Model



```
] dnn_model = Sequential()

imported_model= tf.keras.applications.ResNet50(include_top=False,
        input_shape=(180,180,3),
        pooling='avg',classes=5,
        weights='imagenet')
for layer in imported_model.layers:
    layer.trainable=False

dnn_model.add(imported_model)
dnn_model.add(keras.layers.Flatten())
dnn_model.add(keras.layers.Dense(512, activation='relu'))
dnn_model.add(keras.layers.Dense(5, activation='softmax'))
```

```
] dnn_model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
resnet50 (Functional)	(None, 2048)	23,587,712
flatten_1 (Flatten)	(None, 2048)	0
dense (Dense)	(None, 512)	1,049,088
dense_1 (Dense)	(None, 5)	2,565

Total params: 26,742,673 (102.02 MB)

Trainable params: 1,051,653 (4.01 MB)

Non-trainable params: 23,587,712 (89.98 MB)

Optimizer params: 2,103,308 (8.02 MB)



```
✓ 3s ▶ from tensorflow.keras.optimizers import Adam  
dnn_model.compile(optimizer=Adam(),loss='sparse_categorical_crossentropy',metrics=['accuracy'])
```

```
✓ m [15]  
history = dnn_model.fit(  
    train_set,  
    validation_data=validation_set,  
    epochs=10  
)
```

```
↩ Epoch 1/10  
92/92 ————— 35s 238ms/step - accuracy: 0.6820 - loss: 0.9911 - val_accuracy: 0.8243 - val_loss: 0.4925  
Epoch 2/10  
92/92 ————— 7s 72ms/step - accuracy: 0.8900 - loss: 0.2791 - val_accuracy: 0.8433 - val_loss: 0.4461  
Epoch 3/10  
92/92 ————— 11s 81ms/step - accuracy: 0.9239 - loss: 0.1966 - val_accuracy: 0.8597 - val_loss: 0.4251  
Epoch 4/10  
92/92 ————— 8s 90ms/step - accuracy: 0.9646 - loss: 0.1077 - val_accuracy: 0.8610 - val_loss: 0.4106  
Epoch 5/10  
92/92 ————— 9s 74ms/step - accuracy: 0.9866 - loss: 0.0585 - val_accuracy: 0.8597 - val_loss: 0.4064  
Epoch 6/10  
92/92 ————— 10s 74ms/step - accuracy: 0.9942 - loss: 0.0302 - val_accuracy: 0.8828 - val_loss: 0.3991  
Epoch 7/10  
92/92 ————— 8s 85ms/step - accuracy: 0.9975 - loss: 0.0165 - val_accuracy: 0.8719 - val_loss: 0.4439  
Epoch 8/10  
92/92 ————— 11s 89ms/step - accuracy: 1.0000 - loss: 0.0086 - val_accuracy: 0.8842 - val_loss: 0.4250  
Epoch 9/10  
92/92 ————— 8s 88ms/step - accuracy: 0.9995 - loss: 0.0059 - val_accuracy: 0.8815 - val_loss: 0.4334  
Epoch 10/10  
92/92 ————— 10s 82ms/step - accuracy: 1.0000 - loss: 0.0044 - val_accuracy: 0.8896 - val_loss: 0.4321
```

✓ Making Predictions

```
✓ [16] import cv2
      sample_image=cv2.imread(str(all_sunflowers[1]))
      sample_image_resized= cv2.resize(sample_image, (height, width))
      sample_image=np.expand_dims(sample_image_resized,axis=0)
      print(sample_image.shape)
```

⇒ (1, 180, 180, 3)

```
✓ [17] model_pred=dnn_model.predict(sample_image)
      print(model_pred)
```

⇒ 1/1 ————— 4s 4s/step
[[6.69812918e-01 1.19277752e-06 1.12029806e-01 1.87711090e-01
 3.04449387e-02]]

```
✓ [18] predicted_class=image_cat[np.argmax(model_pred)]
      print("The predicted category is", predicted_class)
```

⇒ The predicted category is daisy